

CURSO DE CIÊNCIA DA COMPUTAÇÃO

Disciplina: Computação Gráfica

Removedor de Manchas em Imagens Digitais com Python

PROPOSTA DE PROJETO — AVALIAÇÃO A3

Nome do(a) Aluno(a) 1: Igor Augusto Araujo Gê — Matrícula: 12823143203

Nome do(a) Aluno(a) 2: Iago Fernandes Teixeira Rosado — Matrícula: 12824126703

Turma: Ciências da Computação

Professor(a): Karla Haryanna Santos Moura e Mikael Rosa de Medeiros

Semestre: 2026.1 | Data de Entrega: 11/06/2026

1. Identificação do Projeto

Campo	Informação
Título do Projeto	Removedor de Manchas em Imagens Digitais com Python
Disciplina	Computação Gráfica
Curso	Ciência da Computação
Tipo de Avaliação	A3 — Projeto Prático Individual
Semestre	2026.1
Linguagem	Python 3.10+
Bibliotecas Principais	OpenCV · NumPy · scikit-image · Matplotlib
Plataforma	Linux / Windows (testado em Ubuntu 22.04 + WSL2)
Versão Final do Código	Beta-v2.20 (detecção multi-escala v2.18 + restauração v2.17)

2. Introdução e Contextualização

A preservação de documentos históricos e a restauração de imagens digitais são desafios recorrentes em arquivística, medicina e entretenimento. Manchas — resultantes de café, oxidação, umidade ou envelhecimento — degradam a legibilidade de textos manuscritos e impressos, dificultando sua digitalização e análise automatizada por OCR (Reconhecimento Óptico de Caracteres).

Este projeto de Avaliação A3 propõe o desenvolvimento de um sistema completo de detecção e remoção de manchas em imagens de documentos, explorando técnicas de Computação Gráfica com OpenCV e NumPy. A abordagem combina análise nos espaços de cor HSV e CIE $L^*a^*b^*$, operações morfológicas e correção de cor multiplicativa para remover manchas preservando o texto subjacente — algo que técnicas clássicas de inpainting não conseguem fazer de forma satisfatória quando a mancha cobre uma área extensa.

O sistema evoluiu por 20 versões incrementais (Beta-v2.1 a Beta-v2.20), cada uma corrigindo problemas identificados por testes com imagens reais. A versão final combina a detecção multi-escala em 4 raios adaptativos (v2.18) com a restauração multiplicativa estável (v2.17).

O estudante deverá demonstrar domínio sobre o pipeline completo de processamento de imagens: detecção baseada em desvio de cor multi-escala, reconstrução por correção multiplicativa, remoção preservando texto e exportação de resultados com métricas quantitativas.

3. Objetivos

3.1 Objetivo Geral

Desenvolver um sistema capaz de detectar e remover automaticamente manchas em imagens de documentos digitalizados, preservando o texto e demais elementos gráficos originais, com validação quantitativa dos resultados.

3.2 Objetivos Específicos

- Implementar pipeline de detecção de manchas usando análise HSV/LAB com critérios multi-escala em 4 raios adaptativos (manchas pequenas e grandes).
- Desenvolver algoritmo de correção de cor multiplicativa que normaliza o tom da mancha em relação à cor estimada do papel, preservando texto e textura.
- Preservar texto e elementos de alta frequência durante a remoção, evitando o borramento causado por inpainting clássico.
- Gerar relatório visual automatizado com imagens de cada etapa e métricas quantitativas (diff_manchas, diff_fora, cobertura).
- Corrigir bugs críticos identificados por testes reais (wrap-around uint8, pixels ciano, referência local de cor errônea).
- Permitir ajuste interativo da sensibilidade de detecção via parâmetro configurável.
- Validar o sistema em imagens reais de documentos manuscritos com manchas de café.

4. Fundamentação Teórica

4.1 Imagem Digital e Espaços de Cor

Uma imagem digital é uma representação matricial composta por pixels, cada um descrito por valores de intensidade ou cor. No modelo RGB (8 bits/canal), cada pixel cobre até 16,7 milhões de cores distintas. A conversão entre espaços de cor é operação fundamental no pré-processamento: o espaço **HSV** (Hue, Saturation, Value) separa matiz da luminosidade, facilitando a detecção de regiões com saturação anormal; o espaço **CIE L*a*b*** permite medir desvios de cromaticidade locais perceptualmente uniformes.

4.2 Técnicas de Processamento de Imagens

Técnica	Descrição	Aplicação no Projeto
Filtro Gaussiano Multi-escala	Suavização em 4 raios adaptativos (11–241 px); elimina texto, preserva cor de fundo da mancha.	Estimativa do fundo da mancha e restauração
Morfologia Matemática	Abertura e fechamento com elemento estruturante elíptico (5×5 e 7×7).	Refino da máscara binária
Detecção LAB Multi-escala	Desvio local ($\Delta L < -3$ AND $\Delta S > 8$) calculado em 4 escalas; score ponderado por raio.	Detecção de manchas pequenas e grandes
Correção Multiplicativa	fator = papel_local / bg_blur [0,5–8×]; pixel_corr = pixel × fator; preserva texto.	Restauração (v2.17, inalterada)
Clamp $B \leq G \leq R$	Papel antigo tem tons quentes. Após correção, B e G são clampados para não ultrapassar R, eliminando pixels ciano/artefatos.	Pós-processamento da correção (v2.16)
Clip [0,255] antes do cast uint8	Evita wrap-around silencioso: np.uint8(262) = 6. Fix crítico da v2.17.	Integridade do canal (v2.17)
Inpainting TELEA	Propagação por Fast Marching Method; eficaz em bordas finas (≤ 2 px).	Suavização de borda da máscara
Distance Transform	Calcula distância à borda da máscara para gerar alpha gradual (0 → 1 em 6 px).	Blend adaptativo nas bordas

4.3 Correção de Cor por Normalização Multiplicativa

A técnica central estima a cor real do papel a partir dos pixels não-manchados com maior luminância (percentil ≥ 70). Um fator de correção por canal BGR é calculado como $fator = papel_local / bg_blur$, onde bg_blur é a imagem original filtrada com kernel Gaussiano adaptativo ao tamanho de cada mancha (9–81 px), eliminando texto. O fator é aplicado pixel a pixel — preservando o texto porque tinta escura $\times$ fator ainda resulta em valor mais escuro que o papel corrigido.

A sanidade da referência local é validada em LAB: se a diferença para a referência global superar 16 unidades ou o eixo b^* diferir em mais de 6 unidades (tom azulado), a referência global é usada — evitando remendos ciano perto de aquarelas ou texto cinza.

5. Metodologia

5.1 Imagens de Teste

- Carta manuscrita digitalizada (*carta.jpg*) com manchas reais de café — resolução $\sim 1280 \times 960$ px, 8 bits por canal, modo BGR.
- Folha de caderno (*folha_caderno.png*) com $\sim 90\%$ da superfície manchada — fundo branco com mancha densa cobrindo a maior parte da página.
- Processamento realizado localmente em Python 3.10+ com OpenCV 4.x e NumPy.

5.2 Ambiente de Desenvolvimento

Componente	Especificação
Linguagem	Python 3.10 ou superior
Ambiente de Execução	IDE local (VSCode) / terminal Linux (Ubuntu 22.04 / WSL2)
Biblioteca Principal	OpenCV-Python (cv2) versão 4.x
Álgebra Matricial	NumPy 1.x
Análise de Cor/Forma	scikit-image 0.21+
Visualização	Matplotlib 3.x

5.3 Pipeline de Processamento (Beta-v2.20)

- **Deteção:** conversão HSV + $L^*a^*b^*$; desvio local ($\Delta L < -3$ AND $\Delta S > 8$) em 4 raios adaptativos; score ponderado; threshold $\rightarrow$ máscara binária; morfologia Open+Close.
- **Separação:** dilatação da máscara 4 px; PNG com alpha=0 nas manchas; imagem de manchas isoladas; máscara repassada à Etapa 3.
- **Reconstrução:** estimativa do papel (percentil ≥ 70); blur adaptativo (9–81 px); fator por canal [0,5; 8,0]; segunda passada de 45% do resíduo; clamp $B \leq G \leq R$; clip [0,255]; TELEA raio 2.
- **Exportação:** 7 imagens diagnóstico + JSON de metadados + log de métricas no terminal.

6. Estrutura de Implementação

6.1 Instalação das Dependências

```
pip install opencv-python numpy scikit-image matplotlib Pillow
```

6.2 Funções Principais (Beta-v2.20)

Função	Finalidade	Biblioteca
_detectar_colorida (img, sensibilidade)	Detecção multi-escala: 4 raios adaptativos, critério $\Delta L/\Delta S$, score ponderado → máscara + score_mapa	OpenCV + NumPy
detectar_manchas (caminho, ...)	Orquestra Etapa 1; salva 01_deteccao.png com overlay de score	OpenCV + scikit-image
separar_manchas (resultado, ...)	Dilata máscara 4 px; gera PNG alpha e imagem de manchas isoladas	OpenCV + NumPy
_corrigir_mancha_colorida (img, mascara)	Ref. local, fator [0,5–8×], clamp $B \leq G \leq R$, clip [0,255] antes do cast	NumPy
reconstruir_imagem (separacao, ...)	Orquestra Etapa 3; chama _corrigir_mancha_colorida	OpenCV + NumPy
remover_manchas (sep, rec, ...)	Aplica máscara reconstruída; gera diff JET ×10 e imagem final	NumPy
exportar_resultados(...)	Salva 7 imagens diagnóstico, JSON de metadados e métricas	OpenCV + Matplotlib
pipeline_removedor (caminho, ...)	Função de alto nível que encadeia todas as etapas	—

7. Evolução do Sistema — Versões e Correções

O sistema passou por 20 versões incrementais. As principais versões, problemas identificados e correções aplicadas estão descritas abaixo:

Versão	Correções / Mudanças Implementadas
v2.1 – v2.4	Detecção inicial HSV+LAB; preenchimento de contornos (elimina anel vazio); detector de manchas claras/translúcidas; auto-detecção do tipo de imagem.
v2.8 – v2.10	Fator multiplicativo ampliado para 8×; segunda passada de empurre residual (45% do caminho restante); expansão de manchas por projeção LAB para cobrir o miolo de manchas grandes.
v2.12 – v2.13	Segunda passada de desvio local com raio grande (~1/5 da menor dimensão) para detectar miolos uniformes. Referência de cor passou a ser LOCAL por contorno, evitando remendos claros demais em regiões mais escuras do papel.
v2.14 – v2.15	Sanidade da referência local: descarta ref. local se distância LAB > 16 ou $b^* < \text{global} - 6$ (indicando tom azulado/cinza), evitando remendos ciano perto de aquarelas ou texto cinza. Limiar geral reduzido de 20 para 16.
v2.16	Clamp de sanidade $B \leq G \leq R$ após correção. Papel antigo tem tons quentes ($B \leq G \leq R$); pixels ciano após correção são sempre artefatos — eliminados independente da causa.
v2.17 ← BUG CRÍTICO	CORRIGIDO: <code>np.clip(corr, 0, 255)</code> ANTES do <code>.astype(uint8)</code> . Sem esse clip, quando <code>papel_local[canal] > 242</code> o teto <code>papel×1,05</code> ultrapassa 255, e o cast uint8 causa wrap-around silencioso (<code>np.uint8(262) = 6</code>). Confirmado em 26 de 41 contornos, produzindo 7.685 pixels ciano. Após o fix: 0 pixels.

Versão	Correções / Mudanças Implementadas
v2.18 – v2.19	Detecção multi-escala com 4 raios adaptativos (critério $\Delta L < -3$ AND $\Delta S > 8$, score ponderado por escala). Tentativas de restauração por inpaint+delta aditivo descartadas por clareamento excessivo em áreas de conteúdo escuro.
v2.20 (VERSÃO FINAL)	Combinação final: DETECÇÃO da v2.18 (multi-escala, 4 raios) + RESTAURAÇÃO da v2.17 (multiplicativa estável). $\text{diff_manchas} \approx 74$ pontos. $\text{diff_fora} = 0,0000$. Texto preservado com variação de apenas 3,8–7,9 pontos.

8. Resultados Obtidos

Ao final da execução, o sistema produz os seguintes artefatos visuais e analíticos:

- **01_deteccao.png** — Máscara com overlay de score (azul proporcional à intensidade) e contornos vermelhos sobre a imagem original.
- **02_frente.png** — Imagem original com canal alpha: alpha=0 nas manchas (transparente), alpha=255 no restante.
- **02_fundo_manchas.png** — Fundo preto com pixels originais de mancha preservados.
- **03_reconstruida.png** — Imagem após correção multiplicativa aplicada às manchas.
- **04_imagem_final.png** — Resultado final restaurado (composição original + reconstruída).
- **04_diferenca.png** — Mapa de calor JET amplificado 10× mostrando onde houve mudança.
- **05_antes_depois.png** — Comparação lado a lado (original × restaurado).
- **relatorio_completo.json** — Metadados por componente e métricas de cada etapa.

Tabela de métricas obtidas:

Métrica	Valor Obtido	Critério / Observação
diff_manchas	~74 pts (v2.20) ~51 pts (v2.17)	Meta: > 40. Detecção multi-escala identifica mais área → maior diff total.
diff_fora	0,0000	Nenhum pixel fora da máscara é modificado.
Cobertura (%)	~10,1%	Máscara dilatada de 123.586 px sobre 1.228.800 px totais.
N. componentes	~89 (v2.20)	Mais componentes detectados graças ao critério multi-escala.
Pixels ciano (pré-v2.17)	7.685 pixels	Causados por wrap-around uint8 (262 → 6).
Pixels ciano (v2.17+)	0 pixels	Após <code>np.clip(corr, 0, 255)</code> antes do cast.
Texto preservado (V < 80, na máscara)	diff = 3,8–7,9 pts	Texto levemente mais claro, porém legível e escuro.

9. Cronograma de Desenvolvimento

Fase / Atividade	Prazo Sugerido	Responsável
Levantamento bibliográfico e estudo de técnicas HSV/LAB	Semana 1	Alunos

Fase / Atividade	Prazo Sugerido	Responsável
Implementação do detector multi-critério (HSV + LAB)	Semana 1–2	Alunos
Implementação do pipeline de inpainting TELEA	Semana 2	Alunos
Desenvolvimento da correção de cor multiplicativa (v2.8–v2.13)	Semana 3	Alunos
Depuração de bugs críticos: ciano, wrap-around uint8 (v2.14–v2.17)	Semana 3–4	Alunos
Implementação da detecção multi-escala (v2.18–v2.20)	Semana 4	Alunos
Testes com imagens reais e ajuste de parâmetros	Semana 4–5	Alunos
Geração automática de relatório visual (7 artefatos + JSON)	Semana 5	Alunos
Avaliação quantitativa e ajuste fino	Semana 5	Alunos
Escrita do relatório técnico e entrega	Semana 6	Alunos

10. Critérios de Avaliação

Critério de Avaliação	Peso (%)	Observações
Documentação da origem da imagem	15%	Deve evidenciar participação ativa dos alunos
Qualidade e variedade das técnicas	30%	8 técnicas distintas implementadas (LAB multi-escala, blur multi-raio, morfologia, correção multiplicativa, clamp $B \leq G \leq R$, clip uint8, inpaint TELEA, distance transform)
Correção e funcionamento do código	5%	Código deve executar sem erros; diff_fora = 0,0000
Análise crítica dos resultados	10%	Discussão técnica dos efeitos aplicados; 20 versões documentadas
Relatório técnico escrito	10%	Formatação, clareza e referências bibliográficas
Apresentação oral / defesa	30%	Domínio do conteúdo e clareza expositiva

11. Entregáveis

Os alunos deverão submeter, em pasta compactada (.zip) ou repositório Git (GitHub/GitLab), os seguintes artefatos:

- **removedor_manchas_v2_20.py** — Código-fonte principal com pipeline completo (detecção, reconstrução, remoção, exportação), comentado em português.
- **requirements.txt** — Lista de dependências Python com versões fixadas.
- **imagem_teste/** — Imagens utilizadas na validação (carta.jpg, folha_caderno.png).
- **resultados/** — Pasta contendo as 7 imagens de diagnóstico e o JSON de metadados gerados na execução.

- **A3_CG_atualizado.pdf** — Este documento de proposta preenchido com os resultados reais obtidos.
- **README.md** — Instruções de execução: dependências, versões Python, parâmetros de linha de comando.

12. Referências Bibliográficas

- [1] GONZALEZ, R. C.; WOODS, R. E. *Processamento Digital de Imagens*. 3. ed. São Paulo: Pearson, 2010.
- [2] BRADSKI, G.; KAEHLER, A. *Learning OpenCV 4: Computer Vision with Python* 3. O'Reilly Media, 2019.
- [3] TELEA, A. An image inpainting technique based on the fast marching method. *Journal of Graphics Tools*, v. 9, n. 1, p. 23–34, 2004.
- [4] SZELISKI, R. *Computer Vision: Algorithms and Applications*. 2. ed. Springer, 2022.
- [5] OpenCV Documentation. Disponível em: <https://docs.opencv.org/> (acesso em 2026).
- [6] NumPy Documentation. Disponível em: <https://numpy.org/doc/> (acesso em 2026).
- [7] scikit-image. Image processing in Python. *PeerJ*, v. 2, e453, 2014.

13. Assinaturas e Validação

Assinatura do(a) Aluno(a) 1

Nome: Igor Augusto Araujo Gê — Mat.
12823143203
Data: 11/06/2026

Assinatura do(a) Aluno(a) 2

Nome: Iago Fernandes Teixeira Rosado —
Mat. 12824126703
Data: 11/06/2026

Assinatura do(a) Professor(a) Orientador(a)

Nome: Karla Haryanna Santos Moura e
Mikael Rosa de Medeiros
Data: 11/06/2026 Nota: _____